DVWA: XSS Reflected e SQL Injection

Test eseguiti al livello di sicurezza LOW

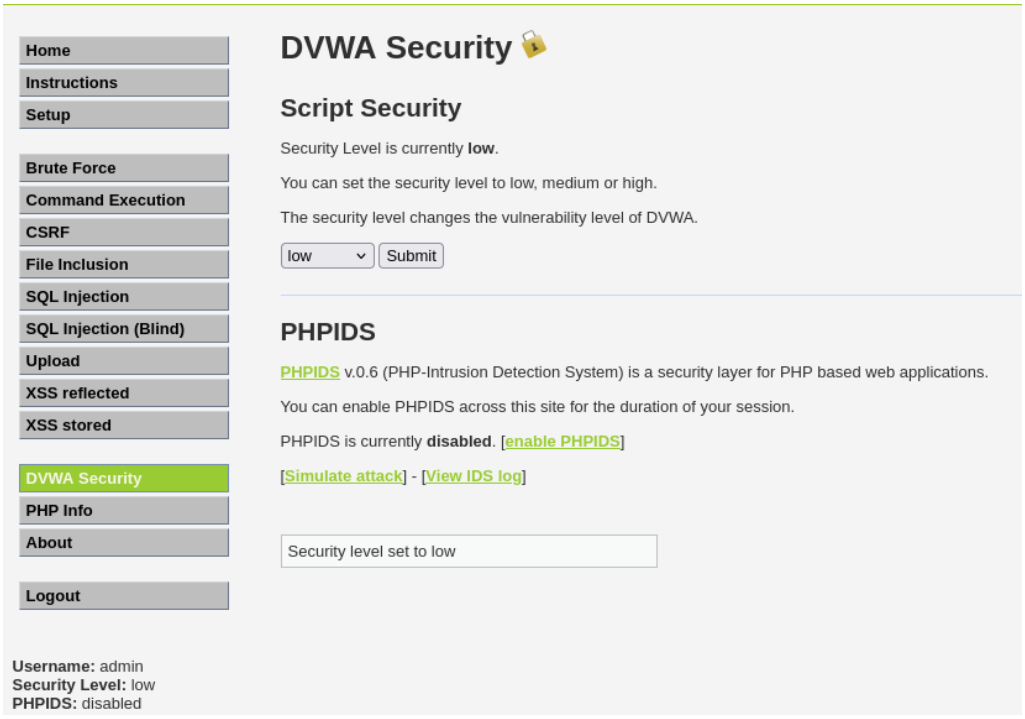
Ambiente	Laboratorio virtuale DVWA / Metasploitable
Target	192.168.60.101
Strumenti	Browser web e strumenti integrati di DVWA
Attività	XSS Reflected e SQL Injection non-blind

Documento basato sui test effettivamente eseguiti e sugli screenshot raccolti.

Obiettivo della pratica

L'obiettivo dell'esercitazione era verificare due vulnerabilità web presenti nell'applicazione didattica DVWA: XSS Reflected e SQL Injection non-blind. La prova è stata eseguita nell'ambiente virtuale predisposto per il corso, con DVWA raggiungibile all'indirizzo 192.168.60.101 e livello di sicurezza impostato su LOW.

Per ogni vulnerabilità sono stati provati prima valori normali e poi input di test. Sono stati inoltre consultati i sorgenti PHP disponibili tramite la funzione View Source, per collegare il comportamento osservato al codice.

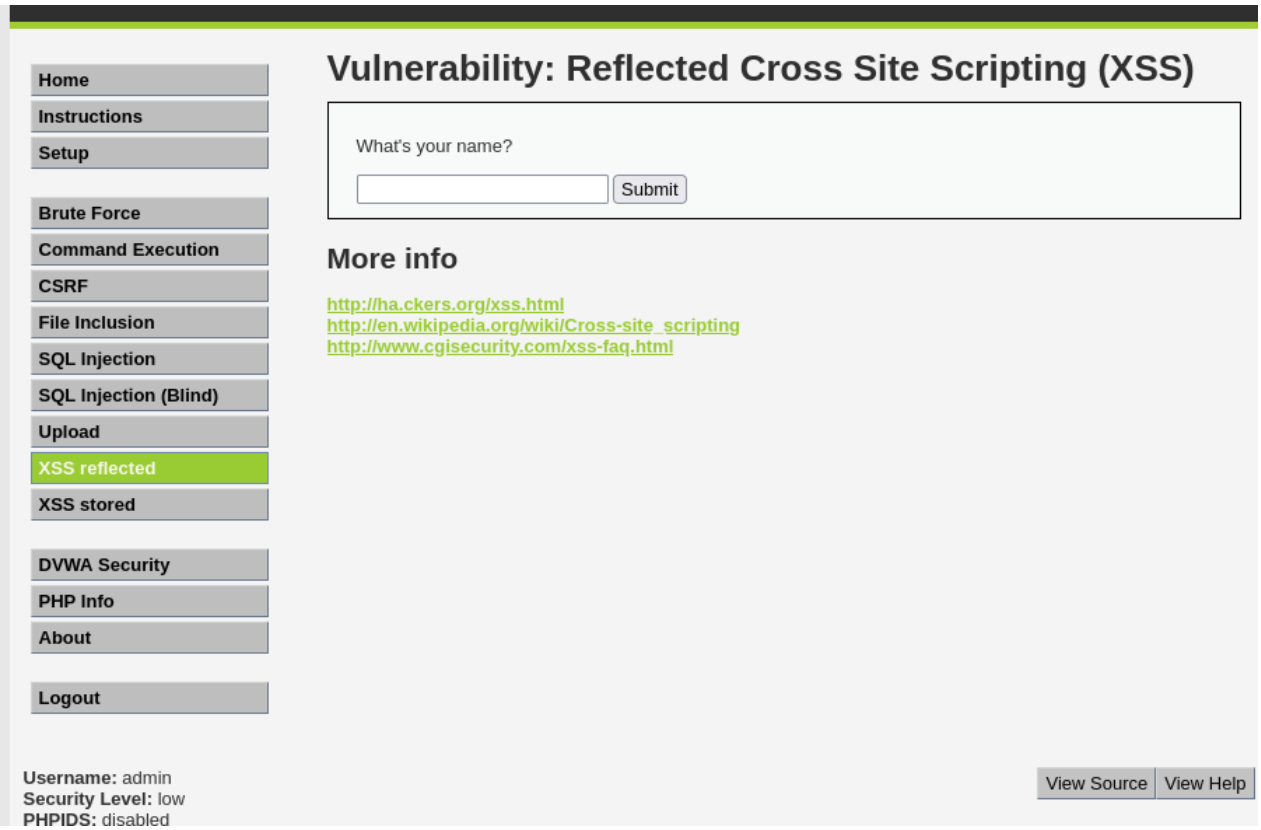


Configurazione DVWA con Security Level impostato su LOW.

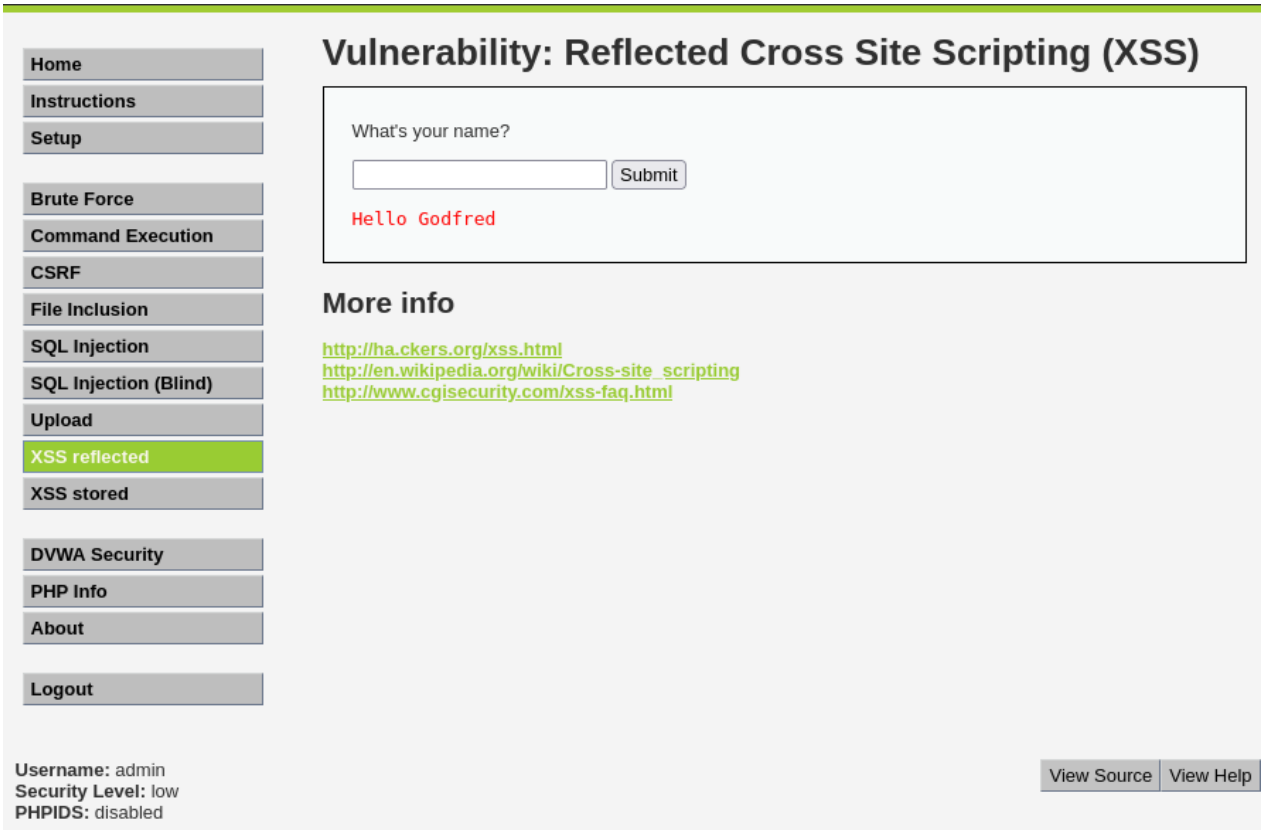
XSS Reflected

Test con input normale

Ho aperto la voce XSS reflected dal menu di DVWA. Il form presenta il campo “What's your name?”, il cui valore viene inviato alla pagina e visualizzato nella risposta. Come prima prova ho inserito un nome normale; la pagina ha restituito un saluto contenente il valore inserito.



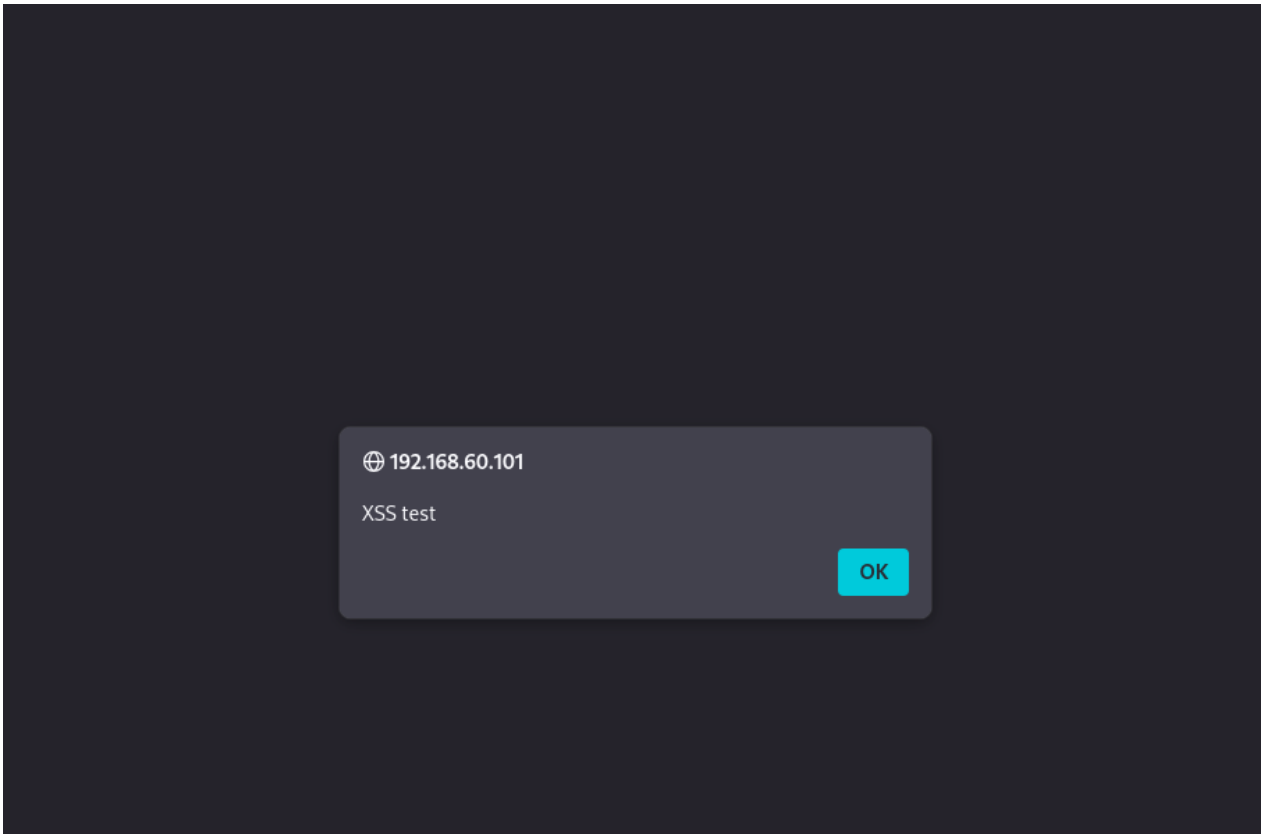
Pagina XSS Reflected prima del test.



Risposta della pagina con un valore normale nel campo nome.

Test JavaScript

Successivamente ho inserito un payload JavaScript dimostrativo nel campo del form. Dopo aver premuto Submit, il browser ha mostrato una finestra di alert con il testo “XSS test”. Questo risultato conferma che, al livello LOW, il valore fornito viene interpretato dal browser come contenuto attivo invece di essere trattato soltanto come testo.



Alert JavaScript “XSS test” eseguito dal browser.

Analisi del sorgente

Nel sorgente Reflected XSS la parte rilevante è:

```
echo 'Hello ' . $_GET['name'];
```

Il parametro name viene letto dalla richiesta GET e concatenato direttamente nella risposta HTML. Nel codice mostrato non è presente una funzione di escaping dell'output, ad esempio htmlspecialchars(). Questo spiega il comportamento osservato: il browser interpreta il markup inserito e, nel test effettuato, esegue JavaScript.

Reflected XSS Source

```php
<?php

if(!array_key_exists ("name", $_GET) || $_GET['name'] == NULL || $_GET['name'] == ''){

 $isempty = true;

} else {

 echo '<pre>';
 echo 'Hello ' . $_GET['name'];
 echo '</pre>';

}

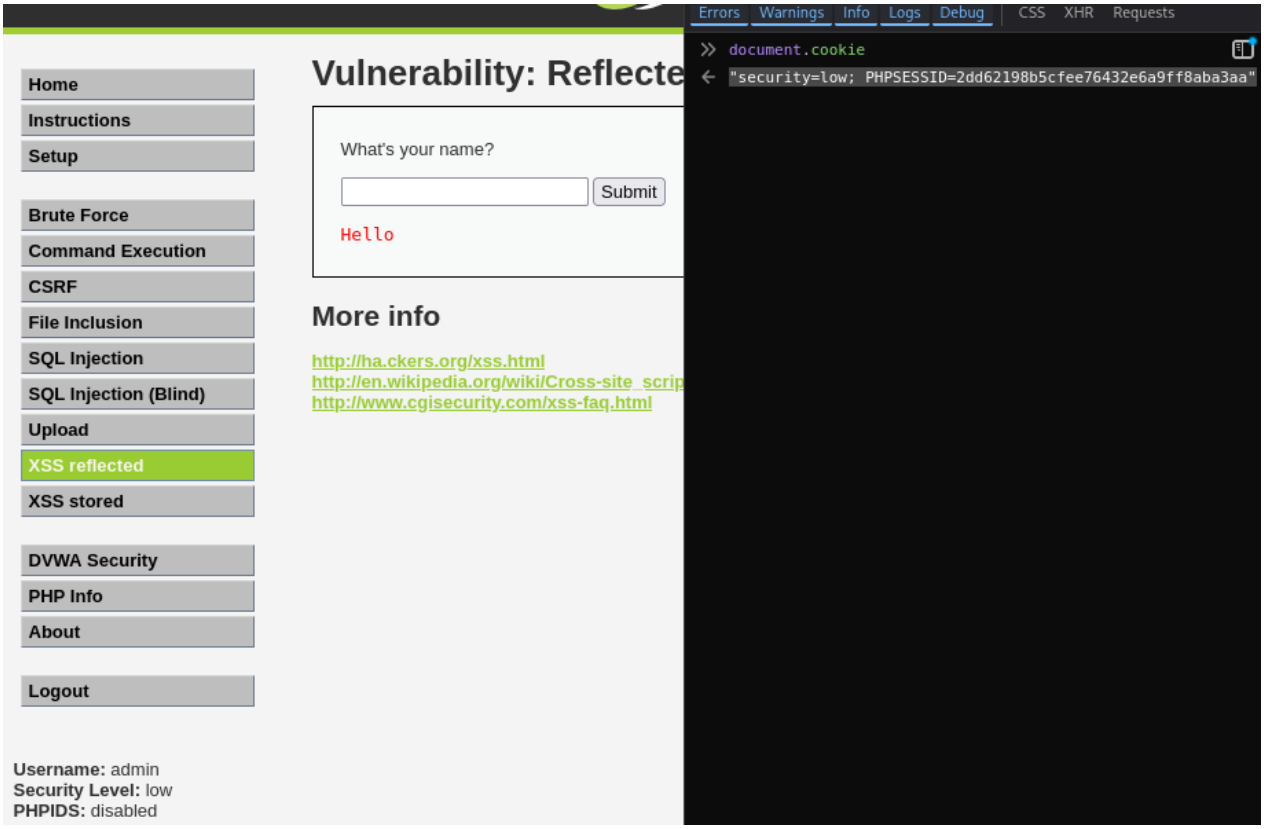
?>
```

View Source di XSS Reflected al livello LOW.

Cookie accessibili dal browser

Per documentare il rischio per la sessione, nella console del browser è stato eseguito document.cookie. L'output visualizzato conteneva il cookie security=low e un PHPSESSID relativo alla sessione del laboratorio. Il valore di sessione non viene trascritto nel report per evitare di esporre inutilmente un identificatore di sessione.

È importante precisare che document.cookie non mostra i cookie protetti dal flag HttpOnly. In un'applicazione reale, un XSS può comunque permettere azioni nel contesto del browser della vittima; la protezione dei cookie con HttpOnly riduce la possibilità di leggerne direttamente il valore tramite JavaScript.

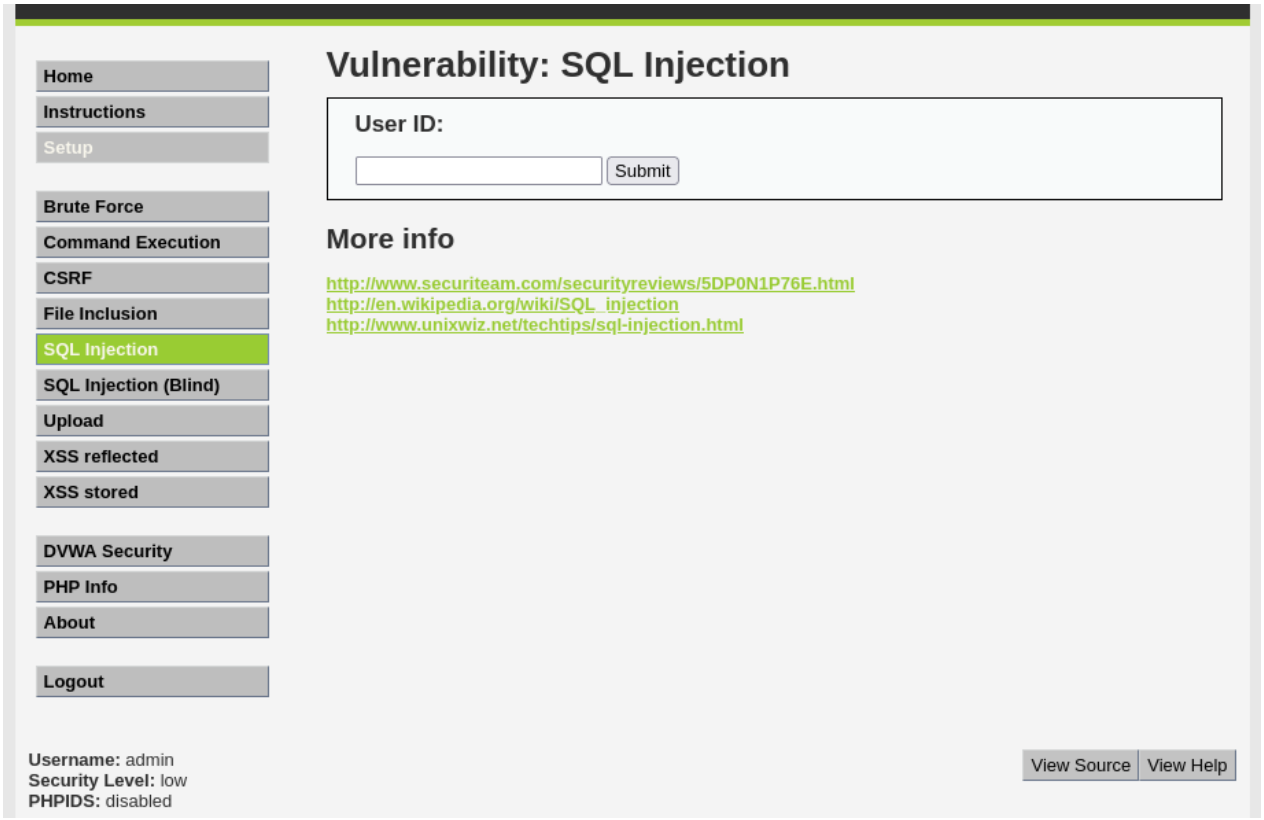


Output di document.cookie nella sessione di laboratorio.

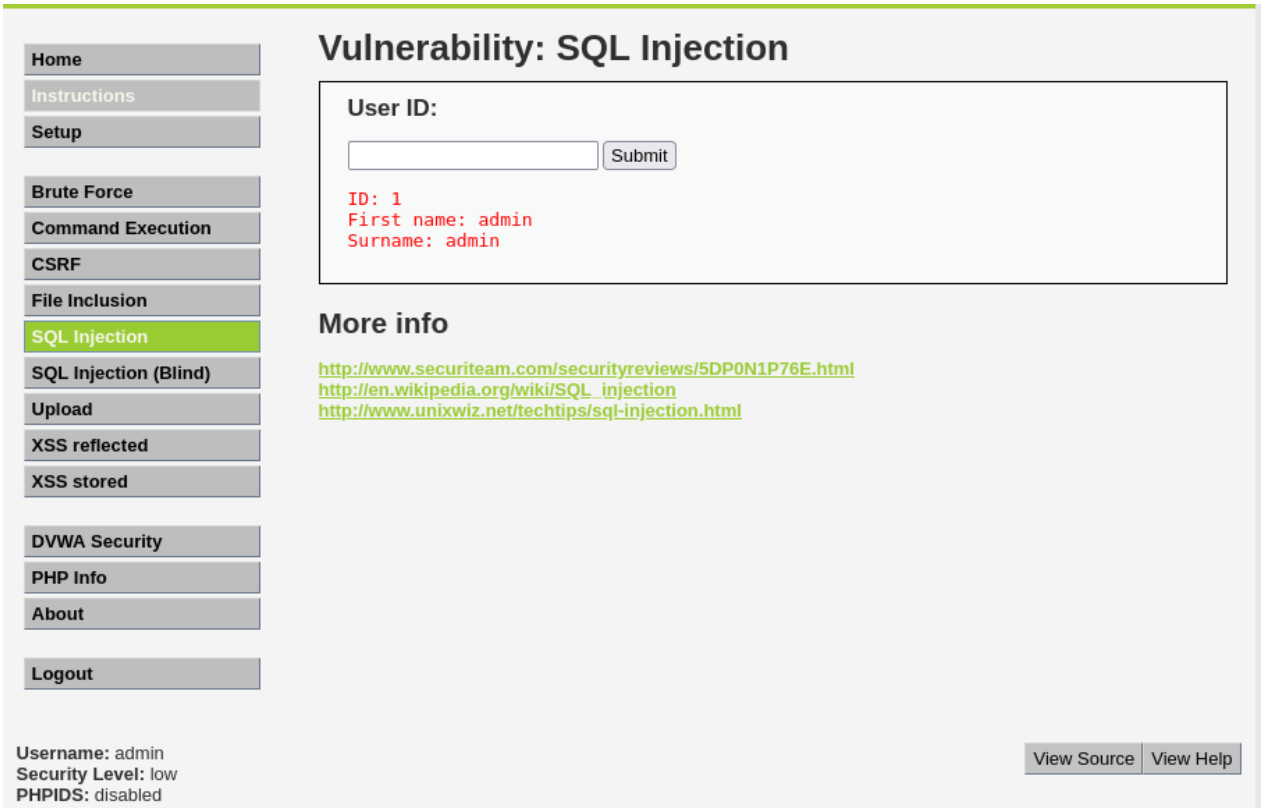
SQL Injection non-blind

Risposta normale

Ho aperto la voce SQL Injection e ho inserito l'ID 1. DVWA ha restituito il record con First name admin e Surname admin. Questo è stato usato come risultato di riferimento prima dei test successivi.



Pagina SQL Injection con il campo User ID.



Risultato normale per l'ID 1.

Analisi del codice PHP

Nel sorgente la query viene costruita in questo modo:

```
$id = $_GET['id'];

$getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

Il valore ricevuto tramite GET viene inserito direttamente nella stringa SQL. Nel codice mostrato non si vede una query parametrizzata né una validazione che limiti l'input a un identificatore numerico. Questa costruzione permette di alterare la logica della query attraverso un input appositamente formato.

SQL Injection Source

```php
<?php

if(isset($_GET['Submit'])){

    // Retrieve data

    $id = $_GET['id'];

    $getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
    $result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>' );

    $num = mysql_numrows($result);

    $i = 0;

    while ($i < $num) {

        $first = mysql_result($result,$i,"first_name");
        $last = mysql_result($result,$i,"last_name");

        echo '<pre>';
        echo 'ID: ' . $id . '<br>First name: ' . $first . '<br>Surname: ' . $last;
        echo '</pre>';

        $i++;
    }
}
?>
```

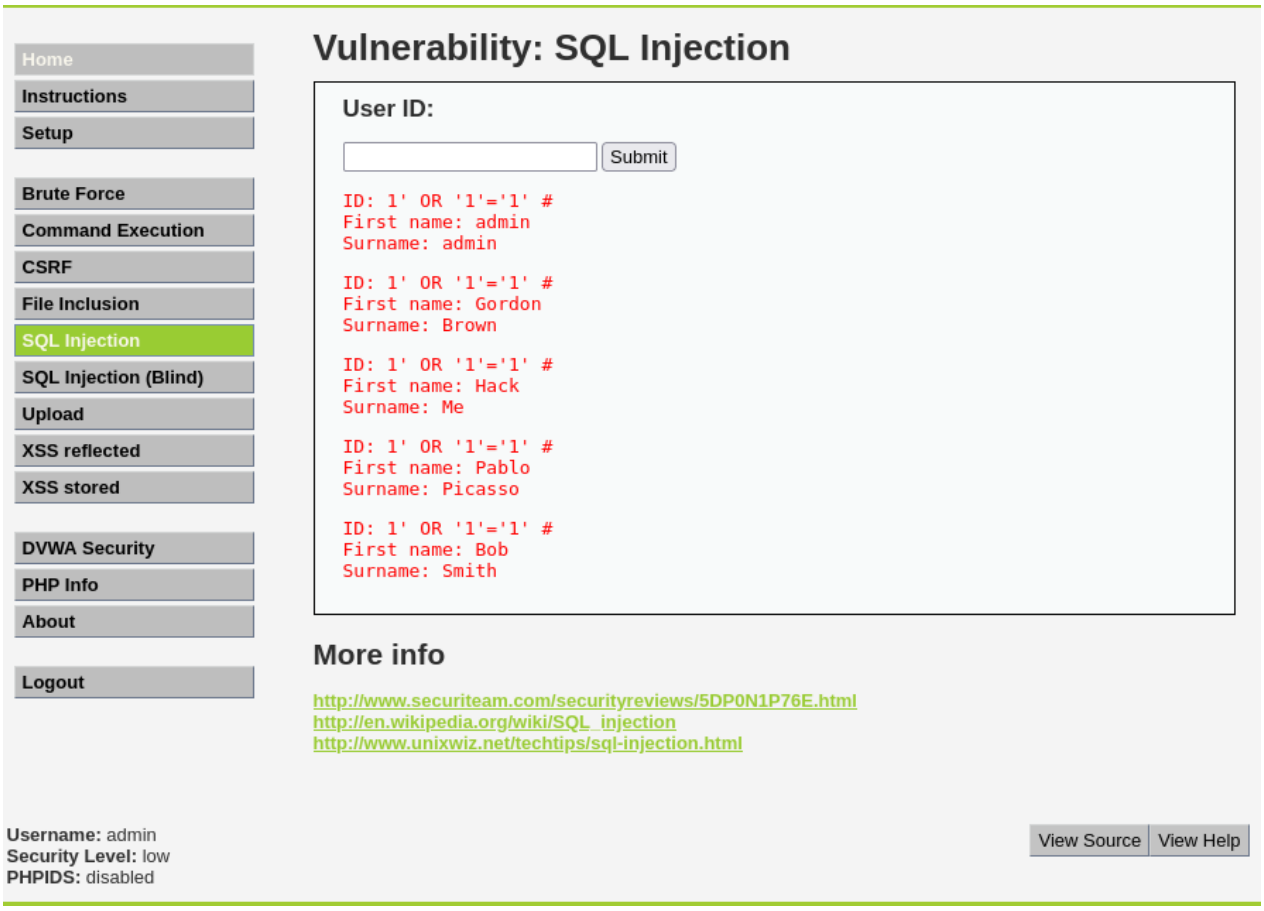
View Source SQL Injection: query SQL costruita concatenando il parametro id.

Test con condizione sempre vera

Ho provato l'input di test 1' OR '1'='1' #. La pagina ha restituito cinque record: admin/admin, Gordon/Brown, Hack/Me, Pablo/Picasso e Bob/Smith. Il risultato è diverso da quello normale, che restituiva un solo record, e dimostra che l'input ha modificato la condizione della query.

```
1' OR '1'='1' #
```

In MySQL la condizione '1'='1' è sempre vera e il carattere # avvia un commento per il resto della riga. Nel laboratorio questo ha fatto sì che la query restituisse tutti i record presenti nella tabella di prova.



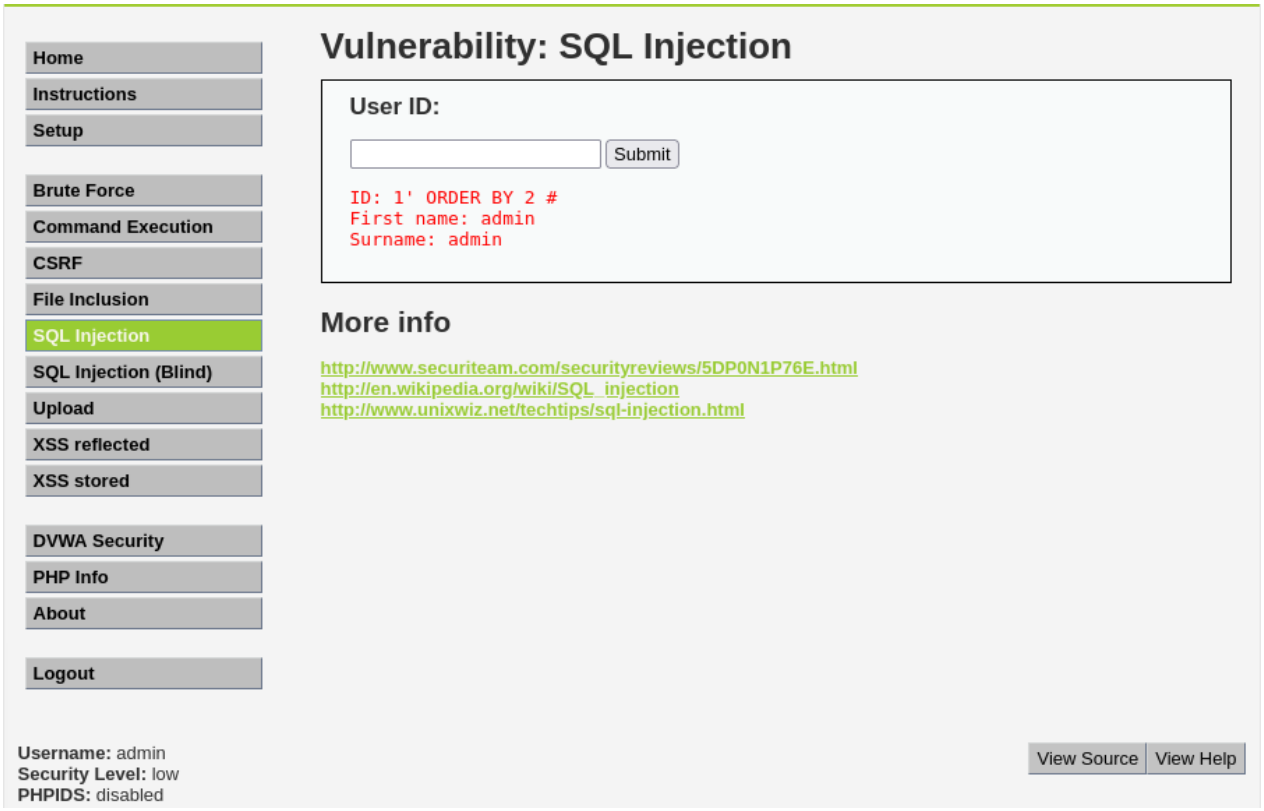
Risultato della SQL Injection: vengono restituiti cinque record.

Verifica del numero di colonne con ORDER BY

Per preparare la dimostrazione UNION ho verificato il numero di colonne della query originale. Il test con ORDER BY 2 ha restituito il record senza errore; il test con ORDER BY 3 ha prodotto il messaggio “Unknown column '3' in 'order clause'”. Questi risultati sono coerenti con la query sorgente, che seleziona due colonne: first_name e last_name.

1' ORDER BY 2 #

1' ORDER BY 3 #



ORDER BY 2: la richiesta viene elaborata e restituisce il record.

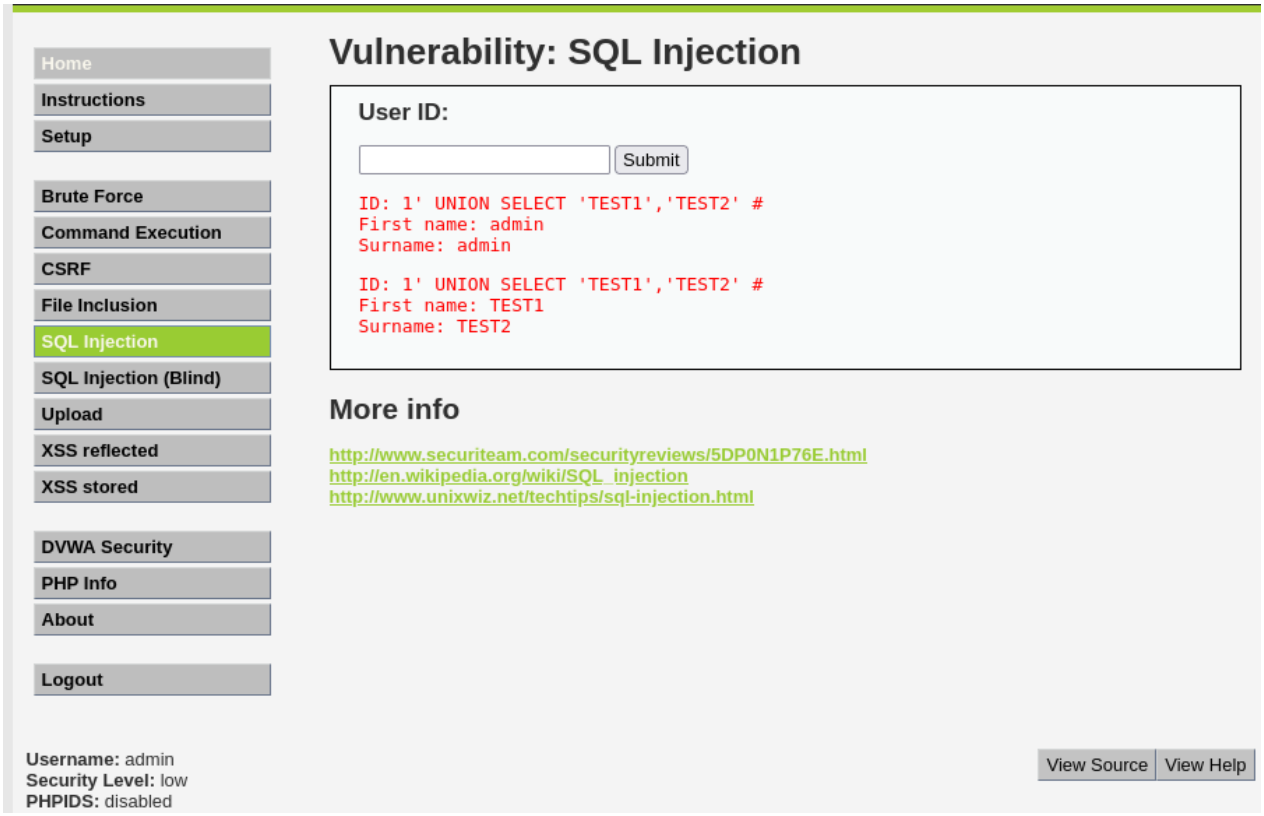


ORDER BY 3: errore che indica che la terza colonna non è disponibile.

Dimostrazione della tecnica UNION

Dopo aver verificato che la query originale restituisce due colonne, ho provato una UNION SELECT di test con due valori testuali. La pagina ha mostrato il record originale admin/admin e un secondo risultato con First name TEST1 e Surname TEST2. Il risultato dimostra che è possibile combinare il risultato originale con quello di una seconda SELECT quando il numero di colonne è compatibile.

```
1' UNION SELECT 'TEST1','TEST2' #
```



UNION riuscita: il risultato aggiunto mostra TEST1 e TEST2.

Sorgente SQL finale

Lo screenshot del sorgente conferma che l'applicazione esegue la query costruita dal parametro id e poi legge i campi first_name e last_name dal risultato. Questo coincide con il comportamento osservato nei test OR e UNION.

SQL Injection Source

```php
<?php

if(isset($_GET['Submit'])){

    // Retrieve data

    $id = $_GET['id'];

    $getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
    $result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>' );

    $num = mysql_numrows($result);

    $i = 0;

    while ($i < $num) {

        $first = mysql_result($result,$i,"first_name");
        $last = mysql_result($result,$i,"last_name");

        echo '<pre>';
        echo 'ID: ' . $id . '<br>First name: ' . $first . '<br>Surname: ' . $last;
        echo '</pre>';

        $i++;
    }
}
?>
```

Sorgente SQL Injection conservato come evidenza finale.

Conclusioni

I test eseguiti a livello LOW hanno confermato entrambe le vulnerabilità previste dalla pratica. Per XSS Reflected, il parametro name viene inserito nella risposta senza escaping e il browser ha eseguito il payload dimostrativo. La verifica con document.cookie ha inoltre mostrato i cookie accessibili allo script nella sessione del laboratorio.

Per SQL Injection, il parametro id viene concatenato direttamente nella query SQL. Il test con una condizione sempre vera ha restituito tutti e cinque i record di prova; i test ORDER BY hanno indicato due colonne; infine, la UNION ha aggiunto correttamente il risultato TEST1/TEST2.

Questa pratica è stata svolta esclusivamente nell'ambiente DVWA predisposto per il laboratorio. Le prove documentano il comportamento osservato al livello LOW; non sono stati eseguiti in questa sessione i test facoltativi a Medium o High.